Calidad y pruebas de software

Introducción al testing automatizado

13 y 14 de octubre de 2026

¿Cómo sabemos que una aplicación hace lo que debería hacer?

Funcionar no siempre significa funcionar bien

Situación

Una aplicación de ventas calcula descuentos y no muestra errores al ejecutarse.

Problema

Al ingresar un descuento del 150 %, el total se vuelve negativo.

¿Cómo detectaríamos el defecto antes de entregar el sistema?

¿Qué significa probar software?

Una prueba compara el comportamiento observado con un resultado esperado.

Entrada

Precio: $1000

Acción

Aplicar un 10 %

Esperado

Total: $900

Manual y automatizado

Prueba manual

Una persona ejecuta acciones, observa lo que sucede y compara el resultado con lo esperado.

Prueba automatizada

El código ejecuta comprobaciones repetibles y comunica si se cumplieron o fallaron.

Son enfoques complementarios.

Los tres niveles de pruebas

Unitarias

Comprueban una función o comportamiento pequeño de manera aislada.

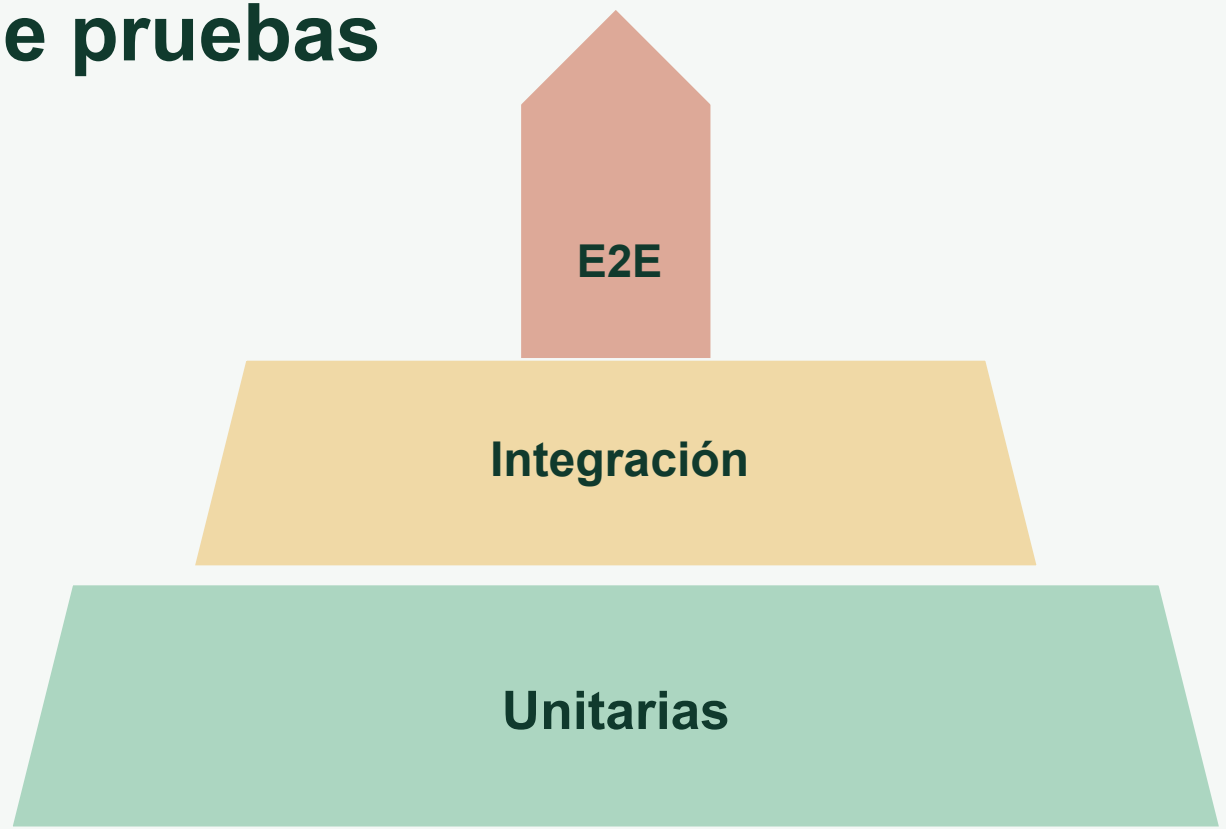
Integración

Verifican que varias partes trabajen correctamente en conjunto.

Extremo a extremo

Recorren un flujo completo desde la perspectiva del usuario.

La pirámide de pruebas



Una orientación para equilibrar rapidez, alcance y mantenimiento; no una regla fija.

Anatomía de una prueba

```
import { it, expect } from 'vitest';

it('aplica un descuento del 10 %', () => {
  const total = calcularPrecioFinal(1000, 10);
  expect(total).toBe(900);
});
```

Preparar → Ejecutar → Comprobar

Casos que conviene comprobar

Habitual

Descuento del 10 %: resultado esperado $900.

Límite

Descuento del 0 %: resultado esperado $1000.

Inválido

Descuento del 150 %: la aplicación debería rechazarlo.

El resultado esperado surge de la regla del negocio.

Cuando una prueba falla

La prueba esperaba que se rechazara un descuento del 150 %.

```
Esperado: RangeError
Recibido: -500
```

¿El problema está en el código, en la prueba o en el requisito?

Primera aproximación a Vitest

Palabras clave

describe: agrupa pruebas.
it: describe un caso.
expect: expresa una comprobación.

Ejecución

npm test

El resultado informa cuántos tests pasaron y cuáles fallaron.

Pruebas para dependencias externas

Stub

Proporciona respuestas predeterminadas.

Mock

Permite controlar o verificar interacciones.

Spy

Registra llamadas para examinarlas.

TDD: probar antes de implementar

1. Rojo

Escribir una prueba que falle.

2. Verde

Implementar lo necesario para aprobar.

3. Refactorizar

Mejorar el código sin romper pruebas.

Es una estrategia de desarrollo, no una condición para poder probar.

Cobertura: un indicador, no una garantía

La cobertura indica qué partes del código fueron ejecutadas durante las pruebas.



Ejemplo ilustrativo de cobertura; no implica ausencia de defectos.

Integración continua (CI)

1. Cambio

Se envía código al repositorio.

2. Verificación

Un flujo instala dependencias y ejecuta pruebas.

3. Resultado

Se informa si las comprobaciones pasan o fallan.

GitHub Actions permite automatizar esta secuencia.

Laboratorio: descubrir y corregir

1. Crear el proyecto e instalar Vitest.
2. Ejecutar dos pruebas iniciales.
3. Agregar un caso inválido.
4. Interpretar la falla y corregir la función.
5. Volver a ejecutar las pruebas.

La guía práctica contiene el código y los pasos completos.

Para recordar

Tres ideas fundamentales

Probar es verificar comportamientos, no solamente ejecutar código.

Un caso inválido puede revelar un defecto que no aparece en el uso habitual.

Automatizar las pruebas facilita detectar regresiones después de un cambio.

Documentación: vitest.dev · docs.github.com/actions